Table of Contents

1. Introduction 1

1.1 Technical Context 3

1.2 Problems Relevance 4

1.3 Justification of Technical Complexity 4

2. Investigation and Application 5

2.1 Quality Model 6

2.2 Quality Process 7

2.3 Mapping to module learning outcome 8

3. System Design, implementation & Testing Strategy 9

3.1 System design 9

3.2 Tools Application & Implementation 9

3.3 Testing Strategy 10

3.4 GitHub Control Evidence 12

3.5 System improvement and problem Solving 13

2.4 Future Improvement Suggestions 16

4. Critical Discussions and Outcomes 18

4.1 Evaluation of Process Improvement 18

4.2 Defect Reduction 18

4.3 Performance Metrics and System Efficiency 19

5. Reflective Statment 20

References 20

Name: Ahmed Al-Soufi

Student No: 15281999

Main Github link: https://github.com/khushi-joshi-05/Food-ordering-website.git

My Github link: https://github.com/aalsofi-uni/ordring-food-webapp-main.git

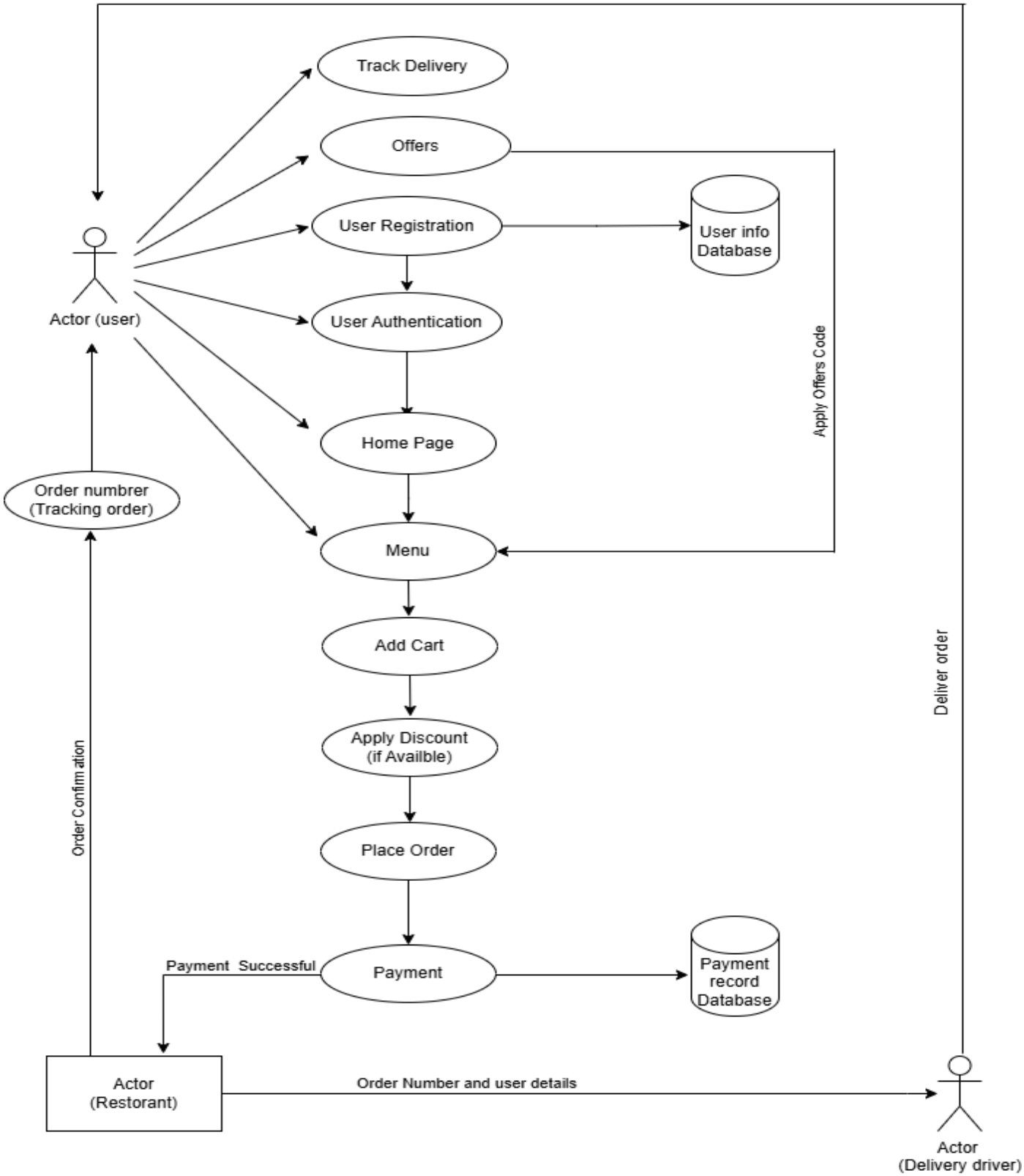
1. System Introduction and Background: -

This is a case study of **Food Ordering Website** which is an open-source project on GitHub. This project has been designed to provide users with a convenient way to order food online and have it delivered directly to their locations or they can just collect it from the restaurant with providing secure users data like (name, phone number, location and etc…).

the website has many pages such as user sign-up where users can register, login where user can access to the website to be able to order food, index page which you can see latest news about the latest news and new dishes, offers page which you can see offers on some particular food and discount codes, menu page where you can find restaurant dishes and food to add in the cart and place an order and contact us page which you can send immediate email to the restaurant through a contact us form in the page.

The purpose this of this case to connect my theory study with a real-life project, so this study focus on how can improve and fix the issues of an existing system by implementing all tools and strategies that I learned during my course module study.

- **Use Case Diagram: -**



1.1 Technical Context: -

This System (Food ordering website) is a web-based that allows customers to browse restaurant menu, place orders, tracking their delivery and make payments online. The system works using a client–server model, where users access the website through a web browser, and the server processes requests and stores data in a database.

- **System Structure**

The system has three main parts:

1. Frontend (User Interface)

This is the most part user will deal with so it must be simple to use.

It is built using web technologies such as HTML, CSS, JavaScript

The frontend allows users to:

- Register and log in
- View restaurants and menus
- Add items to cart
- Place orders

2. Backend

The backend handles the main logic of the system

It is built using JavaScript

The backend is responsible for:

- User authentication
- Managing orders
- Processing payments
- Connecting to the database
- Handling errors and validations

3. Database

The database stores all important data such as:

- User information
- Orders and payment records

- **Development Environment**

  The project was developed using:

  - VS code for version control and code editing
  - Firebase database
  - JSON for data transfer
  - Local server for testing

  The system can be deployed on cloud platforms or web hosting services.

1.2 Problems Relevance: -

The most challenge in online platforms is security such as secure user authentication and account recovery and payment protection. users so often forget their passwords, and poorly implemented reset Methods can lead to security Risks such as unauthorized access or data breaches.

Without a secure authentication and password recovery system:

- Users may give up the platform.
- Customer trust may decrease.
- Businesses may face reputational risks.

This project has some issues. The First thing it needs to make sure that people can login safely. This is called login validation. It also needs to let people reset their passwords using their emails. It needs to store passwords in a safe place online which is called cloud-based credential storage. These things make sure that only the right people can get into their accounts and make changes to them. The project needs to have secure login validation and email-based password reset functionality and cloud-based credential storage to keep accounts safe.

1.3 Justification of Technical Complexity

This System is really complex because it has a lot of parts and important functions like user registration, login menu browsing order placement and order tracking. These functions need to be designed and the website needs to handle data in a secure way and all the different parts need to talk to each other properly.

Firstly, the website has layers like the frontend part that users see the backend part that does code and the database that stores all the information. The frontend part lets users register login, look at the menu, place orders and track their orders. The backend part takes care of what the users ask for makes sure everything runs smoothly and ensures the website works

correctly. The database keeps all the information like user accounts menu items and order details. All these layers need to work in a way that is efficient, secure and reliable.

Secondly, the website needs to be able to handle more the one user at the same time. A lot of customers may register, login and place orders all at the same time. The website needs to make sure that orders are handled correctly and that the order tracking information is updated away. This makes it hard to manage how well the website performs how accurate the data is and how transactions are handled.

Thirdly, the website needs to keep user information safe, like login credentials and personal information. This means the website needs to have login, password protection and proper access control to prevent people from getting in who should not be there and to protect user privacy.

Finally, the food ordering website works with services, like payment gateways and notification systems. This means the website needs to communicate with these services in a way and handle any errors that might happen so that everything works reliably and the food ordering website works smoothly.

2. Investigation and Application of Quality Models and Processes

- **ISO/IEC 25010 Software Quality Model**

ISO/IEC 25010 is an international standard that defines software product quality characteristics. It provides a structured framework for evaluating both functional and non-functional requirements.

The following characteristics are particularly relevant:

- Functional Suitability – Ensures registration, login, ordering, and tracking work correctly.
- Reliability – Ensures the system does not crash during heavily order times.
- Security – Protects user accounts and personal data.
- Performance Efficiency – Ensures fast page loading and quick order processing.
- Usability – Ensures customers can easily navigate the menu and complete orders.
- Maintainability – Allows future updates such as adding delivery tracking features.

- **McCall's Software Quality Model**

McCall's Quality Model focuses on three main areas: product operation, product revision, and product transition. This model helps evaluate how well the system performs and how easily it can be improved.

- Relevant factors for the food ordering system include:

- **Correctness:**
  The system must correctly process user registration, login, and orders. Incorrect processing could result in wrong orders or system errors.
  - **Integrity:**
    The system must protect user data from unauthorised access. This ensures customer privacy and data security.
  - **Usability:**
    Users must be able to easily interact with the system. Simple navigation and clear interface design improve usability.
  - **Maintainability:**
    The system must be easy to update, for example when adding new menu items or improving system features.
  - **Flexibility:**
    The system should allow future improvements such as adding online payment or delivery tracking features.

2.1 Applying ISO/IEC 25010 Quality Model on the System: -

Characteristic	weakness	Actions
Functional Suitability	• User can't register • User can't login	A database was created to store user information, and authentication was implemented to allow users to register and log in securely.
Reliability	• System fails to deliver correct information • No confirmation after finish food order	Order store in the database confirmation email send with order number
Security	• Passwords are not encrypted • Secure payment gateway	Authentication applied to password Emails and payment
Compatibility	• UI crashes when access from different Devices	Css code applied now the system doesn't crash when use from any device
Maintainability	Passed	No action needed
Performance	• There was a preloaded video before redirect to any page was set for 3 seconds	The timing reduced to 1.5 sec now the pages load faster
Usability	passed	No action needed

- **Why ISO/IEC 25010 Software Quality Model was chosen for the Project**

  ISO 20510 was chosen for this System because the food ordering system relies heavily on user access and handles confidential user information, such as personal details and login credentials. Security is one of the most important characteristics of ISO 20510, as it ensures that sensitive data is properly protected from unauthorized access. By following this standard, the system can maintain data confidentiality, integrity, and reliability. This helps ensure that user information is secure and that the system operates safely and efficiently**.**

2.2 investigation of Quality process using Agile: -

- **Introduction of Agile Quality Process: -**

  The Agile quality process is about building and improving the system step by step. It makes sure that the system is good from the start and stays that way throughout the development process.

  Agile does not wait until the system fully built. Instead, it makes sure that quality is part of every stage of the development.

  In Agile development, the work is divided into small parts called **sprints**. Each sprint is a short cycle where I plan the work, develops the features, tests them, and reviews the results. Testing is done continuously, not just when the project is finished. I perform different types of testing, such as unit testing, integration testing, and system testing This way I can find and fix problems on so they do not become big issues later.

  So Agile makes the system better by testing all the system stages. This means that the system is more reliable, works and is usable, for the users. The Agile process helps make the system that's good and that people will like to use.

- **Agile Sprints Breakdown for the Food Ordering website System: -**

  Sprint 1 – Authentication

  - User registration
  - Login system
  - Password validation
  - Basic Firebase setup

Sprint 2 – Ordering System

- Browse menu
- Add to cart
- Place order

Sprint 3 – Payment Integration

- Payment gateway connection
- Transaction validation

Sprint 4 – Tracking & Delivery

- Real-time order status
- Delivery updates

2.3 Mapping to Module Learning Outcomes (1 & 2)

Section of Work	Evidence Provided	MLO 1: Research Depth & Critical Analysis	MLO 2: Application to System
Investigation of ISO/IEC 25010	Explanation of quality characteristics (reliability, security, usability, etc.)	Shows understanding of an international quality model and explains its strengths and weaknesses.	Applied characteristics to the system features (payment, tracking, login security)
Investigation of Agile Methodology	Explanation of step-by-step development, sprint cycles, continuous improvement	Analyses advantages and limitations of Agile as a quality process	Applied sprint structure to authentication, ordering, payment, and tracking modules
Critical Comparison of Models	Comparison between ISO (product-focused) and Agile (process-focused)	shows critical evaluation and synthesis of two quality approaches	Justifies integration of both models to improve system robustness
Identification of Weaknesses	Analysis of tracking instability and payment validation issues	Shows analytical evaluation of system quality gaps	Proposed improvements implemented in subsequent Agile sprints
Quality Improvements	Error handling, validation checks, encryption, performance optimisation	Shows applied research into quality enhancement techniques	Direct implementation within the food ordering website

3. System Design, implementation & Testing strategy

3.1 System design

The food ordering website system looks like what it is made of and how it works. The food ordering system was made so that users can sign up log in look at food items place orders and manage their accounts in a simple way. The design of the system is focused on making it easy to use keeping it safe and making it work well so that users have an experience.

The system is set up with a client-server architecture, where the client side's what the user sees and the server side manages the database. Users interact with system through the user interface, which sends requests to the server. The server then does what the user asked for. Sends back the right answer.
The main parts of the food ordering system are:

- User Interface: this is what lets users talk to the food ordering system.
- Application Layer: this is what handles the stuff like checking if a user can login and processing orders.
- Database: this is where the system stores information, about users, login credentials and order details.
- Authentication System: this makes sure that only people who are allowed to can get into their accounts.

This structured design helps make sure the system is organized, safe and easy to take care of.

3.2 Tool Application and Implementation

The project was improved to be better with the using of tools. This was done to make sure the food ordering website works properly and does not fail. HTML was used to improve the structure of the website. CSS was used to make the website look good and easy to use. JavaScript was used to make the website work the way it should in frontend and backend. This includes things like how users interact with the website checking if someone is logged-in and processing orders. HTML, CSS and JavaScript all work together to make the website that people can use easily.

Visual Studio Code was used to write and test the code for the website. It is a tool that helps me to write code, find mistakes and add new features. This makes it easier to write code and reduces errors.

I also used GitHub to keep track of all the files for the project. My account on GitHub is aalsofi-uni. This helps me keep everything organised and make sure I do not lose any of my work. I can also see what changes I made and go back to a version if I need to.

When I was improving the website, I did it step by step. This means I made one part of the website work then tested it and then made it better. Then I did the same way with the next part. This way the website is reliable can handle a lot of users and's easy to fix if something goes wrong. The food ordering website and its components, such as user interaction and order processing were all improved and tested one, by one. This approach helps make sure the food ordering website is good and works well.

3.3 Testing Strategy

The testing strategy ensures that the food ordering website system meets functional and non-functional requirements including reliability, security, performance, and usability. The system uses a combination of automated and manual testing techniques, aligned with best practices and the software testing pyramid.

- **Testing Objectives: -**
  - Verify correct order placement and discount calculations
  - Ensure secure user authentication
  - Validate Firebase database operations
  - Confirm performance under peak load
  - Prevent security vulnerabilities
  - Ensure a smooth user experience

- **Automated Testing**

  Automated testing was used to improve testing accuracy and efficiency. It helps identify errors quickly and ensures consistent system performance.

- **Unit Testing**

  To verify individual components and functions.

  Tools:

  - JavaScript testing framework.

  Tested Components:

  - Discount calculation
  - Cart total computation
  - Subscription validation logic

- **Integration Testing**

  Integration testing was performed to ensure different system components work together correctly

  Tested Integrations:

  - Firebase Authentication ↔ Firestore
  - Order placement ↔ Database storage
  - Offer application ↔ Cart update
  - Login session persistence

- **System Testing**

  To validate the complete workflow of the application.

  End-to-End Scenarios:

  - User register → login → view menus → applies offer → place order → tracks delivery.

- **Manual Testing**

  - Exploratory testing
  - Usability testing
  - Cross-browser compatibility
  - Edge case validation

- **Security Testing**

  Security is critical for online food ordering systems.

  Testing Areas:

  - Authentication security
  - Role-based access control
  - Firebase security rules
  - Input validation
  - Protection against unauthorized database access

- **User Acceptance Testing**

  - First time registration
  - Applying coupon discount Code
  - Applying promotional offers
  - Tracking delivery status

3.4 GitHub Version Control Evidence.

I used GitHub to store and manage the food ordering website project. The project is saved in my GitHub repository which is called aalsofi.uni. I uploaded all the project files and source code to this repo.

When I was working on the project, I made updates to add new things fix issues and improved the project.

GitHub keeps a record of all the changes I made to the project. This is really helpful because it gives me a backup of all my project files. This means I do not have to worry about losing my work.

Using GitHub helps me keep my project organised. I can also easily go back to versions of my project if I need to by creating branches.

I have included some screenshots of my GitHub repository from my aalsofi-uni account in the coming sections of my report. These screenshots show that I really did use GitHub to manage and support my food ordering website project. GitHub was very useful, for my food ordering system project.

3.5 System Improvement and Problems Solving

- **Sign-up Page**

1. The Project has no Database so a cloud Database on Firebase has been created and a table as well for registering users (table name), the table has 4 columns (Name, email, Username and creation date) as the following: -

```
<script type="module">
  import { initializeApp } from "https://www.gstatic.com/firebasejs/10.7.1/firebase-app.js";
  import { getAuth, createUserWithEmailAndPassword } from "https://www.gstatic.com/firebasejs/10.7.1/firebase-auth.js";
  import { getFirestore, doc, setDoc } from "https://www.gstatic.com/firebasejs/10.7.1/firebase-firestore.js";

  const firebaseConfig = {
 apiKey: "AIzaSyBT3IH3TamNFw0WKeL-F3VotogL7on8f-Q",
  authDomain: "foodi-uni.firebaseapp.com",
  databaseURL: "https://foodi-uni-default-rtdb.firebaseio.com",
  projectId: "foodi-uni",
  storageBucket: "foodi-uni.firebasestorage.app",
  messagingSenderId: "488970239781",
  appId: "1:488970239781:web:b3a9d5f600243d875ba4cc",
  };
```

Connecting with the cloud database (Firebase).

```
document.getElementById("signupForm").addEventListener("submit", async (e) => {
  e.preventDefault(); // stop page reload

    const name = document.getElementById("signup-name").value;
    const email = document.getElementById("signup-email").value;
    const username = document.getElementById("signup-username").value;
    const password = document.getElementById("signup-password").value;
```

Define the elements

2. This part of the code to insert the input text to the database and make sure that all fiels are filled, the user not already exist and the password is a complix.

```
//  Password Complexity
    const strongPassword = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&]).{8,}$/;

    if (!strongPassword.test(password)) {
      alert("Password must be at least 8 characters long and contains:\n* Uppercase letter\n* Lowercase letter\n* Number\n* Special character");
      return; // stop execution
    }

    try {
      const userCredential = await createUserWithEmailAndPassword(auth, email, password);
      const user = userCredential.user;

      await setDoc(doc(db, "users", username), {
        name: name,
        email: email,
        username: username,
        createdAt: new Date()
      });

      alert("Signup successful!");
      window.location.href = "login.html";

    } catch (error) {
      alert("Signup failed: " + error.message);
    }
```

Creating users table and inseritig users data to the database.

- **Login page**

1. I removed username input I just kept the email and the password inputs as a login Credential, I added login code to check if the user credential is already registered to redirect him to Index page before redirect the user it saves user email in the session so user no need to login when place an order.

```
<script type="module">
  import { initializeApp } from "https://www.gstatic.com/firebasejs/10.7.1/firebase-app.js";
  import { getAuth, signInWithEmailAndPassword } from "https://www.gstatic.com/firebasejs/10.7.1/firebase-auth.js";

  const firebaseConfig = {
    apiKey: "AIzaSyBT3IH3TamNFw0WKeL-F3VotogL7on8f-Q",
    authDomain: "foodi-uni.firebaseapp.com",
    projectId: "foodi-uni",
    storageBucket: "foodi-uni.appspot.com",
    messagingSenderId: "488970239781",
    appId: "1:488970239781:web:b3a9d5f600243d875ba4cc",
  };
```

Connecting with the cloud database (Firebase).

2. Reset password has been fixed and became more secure, now you can reset the password by click on forgot password to get an email to allow you do the new password by adding the following code: -

```
const resetBtn = document.getElementById("reset-btn");
  const emailInput = document.getElementById("reset-email");
  const message = document.getElementById("message");

  if (resetBtn) {
    resetBtn.addEventListener("click", async () => {

      const email = emailInput.value.trim();
if (!email) {
    message.style.color = "red";
    message.textContent = "Email is required.";
    return;
  }

  if (!emailInput.checkValidity()) {
    message.style.color = "red";
    message.textContent = "Please enter a valid email address.";
    return;
  }


      try {

        await sendPasswordResetEmail(auth, email);

        message.style.color = "green";
        message.textContent =
          "If this email is registered, a reset link has been sent.";

        emailInput.value = "";

      } catch (error) {

        message.style.color = "green";
        message.textContent =
          "If this email is registered, a reset link has been sent.";

        console.error(error);
      }

    });
  }

});
```

- **Contact us page**

1- User now can contact with the restaurant by email to send any Enquiry, so I added JavaScript code integrated with Emailjs cloud by creating an account in Emailjs cloud website (there also must create your email template) the Emailjs account must be connected with email you want to use it in contact us form.

```javascript
emailjs.init("npcEIBGLCvNUKmJ-Y");

  const form2 = document.getElementById("contactForm2");

  form2.addEventListener("submit", function(event) {

    event.preventDefault(); // STOP PAGE REFRESH

    const name2 = document.getElementById("contact-username").value;
    const email2 = document.getElementById("contact-email").value;
    const message2 = document.getElementById("contact-message").value;

    if (!name2 || !email2 || !message2) {
      alert("Please fill all fields");
      return;
    }

    emailjs.send(
      "service_iaotbjf",
      "template_aiuupch",
      {
        from_name: name2,
        from_email: email2,
        message: message2

      }
    )
    .then(function() {
      alert("Email sent successfully!");
      form2.reset();
    })
    .catch(function(error) {
      alert("Email failed!");
      console.log(error);
    });

  });
```

Define the elements

Service ID and Template ID Can find them in Emailjs cloud

- **Cart Page**

1- this page has more than one function like editing the cart, applying a discount and processing the order, so a table was created in the database (firebase) for the orders to store user orders information history in the following js code created table and store data: -

```
try {

        // Save order
        await db.collection("orders").add({
          orderNumber: orderNumber,
          userUID: user.uid,
          userEmail: user.email,
          checkoutType: checkoutType,
          items: items,
          totalAmount: totalAmount,
          createdAt: firebase.firestore.FieldValue.serverTimestamp()
        });
```

2- In this part if the order was success a confirmation email will be sent to your email To confirm your order and show the order number: -

```
// Send email
        await emailjs.send(
          "service_iaotbjf",
          "template_82bvbdb",
          {
            order_number: orderNumber,
            to_email: user.email,
            checkout_type: checkoutType
          }
        );

        console.log("Email sent");

        alert("Order placed successfully!\nOrder ID: " + orderNumber);
        // Clear cart
localStorage.removeItem("cartItems");
document.getElementById("cart-items").innerHTML = "";
document.getElementById("cart-total").innerHTML = "";

const counter = document.querySelector(".cart-subscript");
if(counter){
  counter.textContent = "0";
}
        // Redirect
        if (checkoutType === "orderOnline") {
        window.location.href = "ordertracking.html?order=" + orderNumber;
        } else {
        window.location.href = "Index.html";
       }

      } catch (error) {
        console.error("Order failed:", error);
        alert("Something went wrong");
      }

    });

});
```

3- This code for applying a discount code: -

```javascript
 let couponApplied = false;
let discountAmount = 0;
document.getElementById("applyCouponButton").addEventListener("click", function(){

    const code = document.getElementById("inputCode").value.trim();

    if(code === "qN6FVAn4"){

        if(couponApplied){
            alert("Coupon already applied!");
            return;
        }

        const cartItems = document.querySelectorAll('.cart-item');
        let total = 0;

        cartItems.forEach(item => {
            const price = parseFloat(item.getAttribute('data-product-price'));
            const quantity = parseInt(item.querySelector('.quantity').textContent);
            total += price * quantity;
        });

        discountAmount = total * 0.30; // 30% discount
        couponApplied = true;

        alert("Coupon applied! You got 30% off.");
        updateTotal();

    } else {
        alert("Invalid coupon code");
    }

});
```

- **Other improvements and problems solved: -**

1- Mobile and tablet compatibility was fixed by writing CSS code in each page.
2- All pages have contact us in the footer now you can send an email for contacting from all pages.
3- Pages load was slow after testing the performance becoming faster.

3.6 Future Improvement Suggestions: -

- The system needs to redesign to become master and child pages, the code will be easy to maintain and no need to repeat similar code in each page for example (footer and header the code will be written just once then all pages can share the same header and footer), the current system used individual pages each page needs to repeat the header and footer code.

- Email's local server must be created, so it will be more secure, easy to handle and manages system emails like (confirmations, contact us and etc….).

- Coupon should be applied once for each user.

- Tracking order could be more accurate by providing user's address when user signing-up.

4. Critical Discussion and Outcomes

4.1. Evaluation of Process Improvement

During the Development of the Food Ordering Website system, the initial development approach was built by quickly adding features without following a clear or structured development process. This led to integration challenges and minor recurring defects. After revising the software model to a more structured and modular approach, measurable improvements were observed.

The introduction of:

- Modular architecture (separating frontend, backend, and database)
- Version control with structured branching
- Defined testing checkpoints

resulted in improved development consistency and reduced rework.

Outcome:
The development process became more predictable and controlled. Feature implementation time decreased because developers could work independently on isolated modules without affecting the entire system.

This demonstrates a direct relationship between structured process control and development efficiency.

4.2 Defect Reduction

I worked hard to reduce defects in the system. I used an approach to testing, which included testing individual parts of the code testing how the parts worked together and testing the whole system. I also reviewed the code and used tools to detect Security weaknesses.

I tested the critical parts of the system such as the payment gateway and the order processing very well.

After implementing structured testing (unit testing + integration testing) and strict validation rules:

- Runtime errors were significantly reduced
- API failure rates decreased
- Post-deployment bug reports dropped
- Bug reduced
- Cart and pricing errors reduced
- Payment-related issues decreased significantly

The reduction in defects was not random; it was the result of the following factors like:

- Early-stage testing
- Code review practices
- Improved error handling

This shows that preventing problems early is more effective than fixing them later.

4.3 Performance Metrics and System Efficiency

using metrics to measure the performance of the food ordering website system. It focuses on how the pages took to load and how long it took for the server to respond and how long it took to process the orders. The team used tools to identify performance issues in the system and optimize performance.

I made several changes to improve performance including optimizing database queries reducing the size of API responses and removing unnecessary scripts. I also implemented caching and improved exception handling.

I was able to reduce the page load time. They also improved the order processing efficiency. Reduced the error rate.

Outcome:

The food ordering website system performed better, the system was more efficient. Could handle more traffic. Efforts to improve performance paid off and the system was more stable.

5. Reflective Statement (Professional reflection, learning synthesis, and continuous improvement)

Developing the Food Ordering Website was an experience. I learned a lot from it. I used and learned new skills in a real live project.

During the development process I gained experience with HTML, CSS and JavaScript. I used them to improve and fix the issues of the website. The website works well on devices.

One key thing I learned was to plan before starting to code. At first, I had some problems. I had issues with form validation and JavaScript errors. I also had trouble integrating EmailJS for the contact form and order process. Debugging these issues helped me improve my problem-solving skills. I learned that small mistakes can cause big problems.

The website must work correctly. It must also be easy to use and look good. It must work well on devices. Testing the website helped me find areas to improve. I improved input validation and reduced errors. I also made the design more consistent and the project more secure.

This project helped me work independently. I managed my time well. I kept testing and improving. I became more confident, in debugging code and using third-party services.

Here are some things I plan to improve my skills:

• I will get better at JavaScript. I want to add dynamic features.

• I want to learn backend development by using Node.js or ASP.net.

• I want to create a full-stack food ordering system from scratch.

This project helped me connect theory and practice. It motivated me to keep developing my web development skills. I want to be more advanced.

References: -

Tools	How it used in this Assignment
ChatGPT	Generating the code to use emails by Connecting JavaScript and Emailjs cloud
Draw.io website	To create “use case diagram”.
www.w3schools.com	Help in some part of writing the code